

Document Purpose

- This document serves three functions:
- 1. **Internal roadmap** — technical decisions deferred from V1, with rationale
 - 2. **Partner review** — shared with potential co-founders and strategic integrators to demonstrate architectural depth and honest assessment of limitations
 - 3. **Release control** — defines what ships in V1, what is scoped for V2, and what remains research

This is a living document. Updated weekly alongside the 30-day outreach plan.

V1 Current State — What Is Deployed

Component	Status	Tests	Verification
YeldenVault (ERC-4626)	✓ Complete	182	Certora 7/7, Echidna 3/3
AIAgentRegistry v3	✓ Complete	69	Certora 7/7, Echidna 3/3
ZKVerifier (Groth16)	✓ Complete	16	532 constraints, trusted setup
YeldenDistributor	✓ Complete	12	—
\$YLD Token	□ Planned	—	—
RWA Adapters	□ Planned	—	—
Chainlink DON Integration	□ Planned	—	—

Total: 198 tests passing. 0 failing. MIT License. Open-source.

Critical External Feedback — Incorporated

Feedback Source 1 — Anonymous Senior Engineer (via X, post-Axiom)

Profile: deep ZK + formal verification background, read full repo

Critique: Chainlink DON as SCORER_ROLE is not "code is law" — it is a trusted external service. DON operators can be coordinated or influenced. The initial approval step retains administrative control, creating the same gatekeeping dynamic the Axiom criticizes.

Status: Valid. Documented as known limitation. V2 addresses with VerificationTier architecture.

Feedback Source 2 — BuilderBen (@BuilderBenv1), AgentProof

Profile: AgentProof — 50k+ agents indexed, 143k+ evals, ERC-8004 alignment, UK-based

Critique:

"I'd keep isEligible() as a helper, not the primitive. Expose the raw score and let integrators decide their own thresholds. A bool flattens the signal too much for different use cases."

Status: Correct. Incorporated into V2 interface design below.

Context: BuilderBen is co-building AgentProof — a trust primitive for the agentic economy with significant traction (50k+ agents, 143k+ evals). AgentProof and AI-Agent-Registry address complementary layers of the same problem. Potential integration path: AgentProof as upstream trust oracle feeding SCORER_ROLE inputs into AI-Agent-Registry.

Open Problem 1 — Interface Primitive Design

Current V1 Implementation

solidity

// Current — bool flattens the signal

function isEligible(address agent) external view returns (bool);

// Current — raw score available but not primary interface

function score(address agent) external view returns (uint256);

V2 Proposed Interface

solidity

// V2 — score as primary primitive

```
function scoreOf(address agent) external view returns (uint256 score,  
VerificationTier tier, uint256 lastUpdate);
```

// V2 — isEligible becomes a convenience wrapper only

```
function isEligible(address agent, uint256 minScore) external view returns (bool);
```

// V2 — integrators set their own policy

```
function isEligibleForPolicy(address agent, Policy calldata policy) external view  
returns (bool);
```

```
struct Policy {
```

```
    uint256 minScore;
```

```
    VerificationTier minTier;
```

```
    uint256 maxStaleness; // seconds since last score update
```

```
}
```

Rationale: A protocol managing \$50M RWA positions needs different trust assumptions than a protocol gating access to a \$500 UBI bonus. `isEligible()` as `bool` collapses this distinction. `scoreOf()` + caller-defined `Policy` preserves the full signal.

Impact on Axiom: The Axiom remains intact. Accountability is still encoded in score history. The interface change makes the accountability layer more composable, not less rigorous.

Open Problem 2 — DON Centralization Vector

The Problem

SCORER_ROLE is held by Chainlink DON in the production design. This means:

- Score updates are not trustless — they depend on DON operator integrity
- Initial agent approval requires SCORER_ROLE action — administrative dependency

- Complex behaviors (sophisticated front-running, cross-protocol coordination) require off-chain inference — not pure on-chain derivation

This is the most structurally difficult problem in the design. It cannot be fully solved with current tooling. It can be mitigated with layered trust architecture.

V2 Solution — VerificationTier

solidity

```
enum VerificationTier {
    ON_CHAIN_ONLY,    // Score 0–400: pure on-chain observables
    DON_VERIFIED,     // Score 401–800: DON + cryptographic proof of origin
    DISPUTE_RESOLVED // Score 801–1000: economic dispute + zk-proof
                    resolution
}
```

Tier 1 — ON_CHAIN_ONLY (score 0–400)

Zero external trust. Any protocol can integrate with full trustlessness.

Observable metrics (derived directly from chain state):

- Liquidation execution: liquidationCount / liquidationOpportunities
- Slippage management: actualSlippage / expectedSlippage across N transactions
- Uptime: consecutive blocks with valid keeper activity
- TVL delta: net value change attributable to agent actions

solidity

// Tier 1 score update — permissionless, fully on-chain

```
function updateOnChainScore(
    address agent,
    uint256 liquidationSuccessRate, // from protocol events
    uint256 slippageScore,          // from DEX logs
    uint256 uptimeBlocks            // block range observation
) external {
    require(_verifyOnChainProofs(agent, liquidationSuccessRate, slippageScore,
    uptimeBlocks));
}
```

```

    _updateScore(agent, _calculateTier1Score(...),
VerificationTier.ON_CHAIN_ONLY);
}

```

Tier 2 — DON_VERIFIED (score 401–800)

DON provides off-chain inference with cryptographic proof of data origin. Requires multiple DON consensus (threshold signatures). Covers complex behaviors not derivable from raw chain state.

Tier 3 — DISPUTE_RESOLVED (score 801–1000)

Economic dispute mechanism. Agent that disagrees with score posts additional stake + zk-proof of correct behavior. Resolution via:

- zkML proof of strategy correctness
- On-chain prediction market (Polymarket-style fork)
- Multi-party arbitration with economic bonds

Open Problem 3 — DON Slashing Reversal

The Problem

Current design: DON slashes agent. No recourse mechanism.

If DON errs — misconfigured job, stale data, oracle manipulation — agent loses stake with no appeal. This creates asymmetric risk that rational agents will price into their participation decision, reducing registry adoption.

V2 Solution — contestSlash()

solidity

```

function contestSlash(
    uint256 slashId,
    bytes calldata zkProof,
    bytes calldata publicInputs
) external onlySlashedAgent(slashId) {
    // Verify ZK proof of correct behavior during slash window
    bool proofValid = zkVerifier.verify(zkProof, publicInputs);

```

```

if (proofValid) {
    // Restore agent stake
    _restoreStake(slashId);
    // Penalize SLASHER_ROLE economically
    _penalizeSlasher(slashId);
    // Emit for governance review
    emit SlashContested(slashId, msg.sender, true);
} else {
    // Burn additional stake — false contest is expensive
    _burnAdditionalStake(msg.sender, CONTEST_PENALTY);
    emit SlashContested(slashId, msg.sender, false);
}
}

```

Economic alignment: DON operators face financial loss for incorrect slashing. This creates incentive alignment without requiring trust — the math enforces correct behavior.

Reuses: ZKVerifier.sol already deployed. Groth16 proof verification infrastructure exists. contestSlash() is an incremental addition, not a new system.

Open Problem 4 — Score Bootstrap (Cold Start)

The Problem

New agent. No history. No score. Starting score = 300 (fixed).

Two sub-problems:

1. Fixed 300 treats a proven keeper with 2 years of history identically to a brand-new address
2. 50 \$YLD minimum stake assumes \$YLD exists and has accessible liquidity — pre-launch, this is undefined

V2 Solution A — Dynamic Initial Score via ZK Proof

solidity

```
function registerWithProvenHistory(
```

```

bytes calldata zkProof,    // Proof of prior execution history

bytes calldata publicInputs, // Protocol addresses, block ranges, success rates

uint256 stakeAmount

) external{

    // Verify off-chain history without revealing strategy

    bool valid = zkVerifier.verify(zkProof, publicInputs);

    uint256 initialScore = valid ? _calculateBoostScore(publicInputs) :
    BASE_SCORE;

    _registerAgent(msg.sender, stakeAmount, initialScore);

}

```

Reuses contribution.circom already compiled. Agent proves prior execution history (liquidations on Aave, keeper activity on Compound) without revealing strategy or identity.

V2 Solution B — Genesis Period

- Pre-\$YLD launch: stake minimum = 0, score earned via ZK proof of work only
- Converts to stake requirement when \$YLD launches
- Genesis agents receive founding score that reflects real contribution

Open Problem 5 — Adoption Incentive (Optional vs Substrate)

The Problem

Sophisticated protocols will maintain parallel whitelists. Registry becomes one option among many, not the shared substrate the Axiom envisions.

V2 Solution — Integration Incentive Layer

solidity

// Protocols that integrate receive access to verified agent pool

// Protocols that don't integrate bear full adversarial agent exposure

```

mapping(address => bool) public integratedProtocols;

```

```

mapping(address => uint256) public protocolIntegrationTimestamp;

// Early integrators receive discounted fee rates on agent pool access
function registerProtocol() external {
    integratedProtocols[msg.sender] = true;
    protocolIntegrationTimestamp[msg.sender] = block.timestamp;
    // Early integration = lower platform fee forever (locked at registration time)
}

```

Network effect logic: Each protocol that integrates makes the registry more valuable to every agent. Each agent that registers makes the registry more valuable to every protocol. First-mover advantage is structurally encoded.

AgentProof Integration Path

Potential partnership with BuilderBen (@BuilderBenv1)

AgentProof and AI-Agent-Registry are complementary, not competing:

Layer	AgentProof	AI-Agent-Registry
Focus	Trust primitive, cross-chain indexing	Economic accountability, on-chain punishment
Scale	50k+ agents, 143k+ evals	V1 — registry primitive
Verification	Off-chain eval + ERC-8004 metadata	Formal verification, ZK proofs
Punishment	None (trust layer only)	Slash 10/50/100%, burn to 0x0
Score source	External evals	On-chain behavior + DON

Integration hypothesis: AgentProof scores feed as high-quality inputs into AI-Agent-Registry SCORER_ROLE. AgentProof handles the trust indexing layer. AI-Agent-Registry handles the economic accountability layer. Combined: trust + consequence.

This resolves part of the DON centralization problem — AgentProof's 143k+ evals provide decentralized, high-quality scoring data that reduces dependency on a single DON.

Next step: Technical discussion with BuilderBen on data format compatibility and potential SCORER_ROLE integration.

Release Control

V1 — Ships As-Is

- AI-AgentRegistry v3 with current isEligible() interface
- DON as SCORER_ROLE (documented limitation)
- Fixed score bootstrap at 300
- No contestSlash mechanism

V1 is production-ready for integration. Limitations are documented, not hidden.

V2 — Post Co-founder, Pre-Mainnet

- VerificationTier architecture (3 tiers)
- scoreOf() as primary primitive, isEligible() as helper
- contestSlash() with ZK proof verification
- Dynamic bootstrap score via registerWithProvenHistory()
- Protocol integration incentive layer

V3 — Post-Audit, Post-\$YLD Launch

- zkML for Tier 3 dispute resolution
- EIP proposal for standard agent registry interface
- AgentProof integration (pending partnership)
- Score decay mechanism (time-weighted history)

Research — No Timeline

- Purely on-chain score for complex behaviors (structurally limited — see Problem 2)
- Prediction market for slash disputes
- Cross-chain score portability

Known Limitations — Honest Assessment

These limitations are documented here for partner review. They are not hidden from co-founders or integrators.

1. **DON dependency** — SCORER_ROLE via Chainlink is pragmatic but not trustless. Acknowledged. V2 mitigates with VerificationTier. Full resolution requires on-chain ML or equivalent — not available today.
2. **isEligible() signal flattening** — bool interface loses score nuance. Acknowledged. V2 replaces with scoreOf() as primary primitive.
3. **Fixed bootstrap score** — 300 for all new agents regardless of history. Acknowledged. V2 adds registerWithProvenHistory() with ZK proof.
4. **\$YLD pre-launch stake** — minimum stake assumes token exists. Pre-launch, entry barrier is undefined. Acknowledged. Genesis period with zero-stake ZK-proof-only registration planned.
5. **No slash contestation** — current design has no recourse for incorrectly slashed agents. Acknowledged. V2 adds contestSlash().

None of these limitations affect V1 utility as a registry primitive. They affect the strength of the Axiom claim at scale.

Architectural Principle

Stated for co-founder alignment:

The goal is not to ship the perfect system. The goal is to ship the honest system — one where limitations are documented, where the upgrade path is clear, and where each version is more trustless than the last.

V1 is pragmatic. V2 is rigorous. V3 is the Axiom made real.

Last updated: March 2026 Next review: March 8, 2026 Contact:
yeldenfund@gmail.com github.com/yeldenfund/yelden-protocol